

DentalCloud Production Incident Report

PostgREST Connection Pool Exhaustion and Automatic ONP Request Amplification

Report date	21 September 2026
Incident window	20-21 September 2026
Affected system	DentalCloud production application and self-hosted Supabase data API
Severity	SEV-1 - client-facing data service outage
Current status	Resolved; permanent mitigation deployed and monitored
Triggering change	0a107c6 - Batch automatic ONP patient updates
Corrective change	1b7bc4d - Prevent concurrent automatic ONP refresh storms

Classification: Internal Management Use

Prepared from repository history, production service logs, database diagnostics, recovery records, and post-remediation validation results.

1. Executive Summary

DentalCloud experienced a client-facing data-service outage when PostgREST, the REST gateway used by the application to access PostgreSQL, could no longer acquire connections from its internal pool. PostgreSQL remained healthy, but application data requests returned **HTTP 503** with **PGRST003** and subsequently **PGRST002**.

Management conclusion

The outage was caused by application request amplification interacting with a small PostgREST connection pool. A code change deployed the previous evening converted one large automatic ONP update into many sequential PATCH requests. Routine patient reads could trigger this work, while simultaneous users or tabs could run duplicate full scans and update sequences. The permanent fix replaces that pattern with one atomic, advisory-locked database RPC, adds client-side single-flight and cooldown controls, and adds an independent health watchdog.

Assurance statement

The same specific failure mode is now strongly mitigated at three independent layers: request reduction and locking in the database, duplicate suppression and cooldown in the client, and automatic recovery through a server watchdog. This materially reduces recurrence risk. No production system can responsibly be guaranteed never to fail; unrelated defects, infrastructure faults, provider outages, or future changes can still cause downtime.

Area	Outcome
Service recovery	PostgREST restored without restarting PostgreSQL; Auth and Storage remained available.
Permanent correction	Atomic database RPC, transaction advisory lock, client single-flight, and five-minute cooldown deployed.
Prevention	One-minute three-strike watchdog verifies a real authenticated query and restarts only PostgREST when PostgreSQL is healthy.
Validation	483 tests passed, TypeScript passed, production build passed, and 12 concurrent production RPC calls returned 200.

2. Business Impact

- Clients could reach the frontend, but data-dependent screens and workflows failed because REST database calls returned HTTP 503.
- Authentication and Storage remained healthy; the incident was concentrated in the PostgREST data API.
- No evidence of database corruption or data loss was found.
- A logical database backup was created before remediation, and the existing daily backup passed integrity validation.

3. Scope and Affected Components

Component	State	Finding
Frontend / Netlify	Reachable	HTTP 200; application shell loaded.
Nginx / Kong	Reachable	Proxy path available and returning upstream responses.
Supabase Auth	Healthy	Health endpoint returned HTTP 200.
Supabase Storage	Healthy	Status endpoint returned HTTP 200.
PostgreSQL	Healthy	27/100 sessions, no blockers, no idle-in-transaction sessions.
PostgREST	Failed	Pool timeouts and schema-cache failures; HTTP 503.

4. Incident Timeline

Time (Myanmar, UTC+06:30)	Event
20 Sep, 19:38	Commit 0a107c6 changed automatic ONP updates from one bulk PATCH to URL-safe batches of 20.
20 Sep, 22:42	First confirmed PGRST003 pool acquisition timeout appeared in production logs.
21 Sep, morning	Client-visible outage reported. REST requests returned HTTP 503 while the host, proxy, Auth, and Storage remained reachable.
21 Sep, 11:04	Logical backup completed and checksummed. Only supabase-rest was restarted.
21 Sep, 11:05	PostgREST rebuilt its schema cache and real REST table queries returned HTTP 200.
21 Sep, 11:12	PostgREST watchdog and hardened backup permissions activated.
21 Sep, 11:23	Corrective commit 1b7bc4d created and pushed to origin/main.
Post-fix	Atomic RPC deployed and tested with 12 simultaneous calls; all returned HTTP 200 with no PGRST002/PGRST003 recurrence.

5. Technical Root Cause

PostgREST had a default pool size of ten database connections. The application tied automatic data maintenance to routine patient retrieval. The last commit changed the write phase from one oversized update into multiple PATCH requests, solving proxy request-line limits but increasing request volume and duration.

Request amplification before correction

For N eligible patients, one refresh could generate approximately:

```
1 settings query + 1 full eligibility scan + ceil( $N / 20$ ) PATCH requests + the requested patient read
```

Full patient retrieval was used by startup, dashboards, exports, assistant loading, patient profile loading, and refreshes. Multiple users or tabs could overlap the same work because there was no shared lock, single-flight guard, or cooldown.

Category	Finding
Primary cause	Automatic ONP maintenance was triggered from read paths and expanded into many PostgREST PATCH requests.
Triggering change	Commit 0a107c6 introduced sequential batches of 20, increasing request count per conversion.
Concurrency gap	No guard prevented multiple clients or tabs from running the same conversion.
Capacity factor	A ten-connection PostgREST pool provided limited headroom for overlapping sequences.
Detection gap	No real-query watchdog automatically recovered a running but unhealthy PostgREST service.

Why PostgreSQL looked healthy

PostgreSQL accepted direct connections and had remaining capacity. The failure existed in PostgREST pool and schema-cache state, so database process health alone did not detect the client-facing outage.

6. Code Defect Analysis

Before permanent fix	Risk
Routine patient reads invoked automatic maintenance.	A read unexpectedly created writes and extra latency.
Eligible patients were converted in batches of 20 PATCH requests.	Request count grew linearly with eligible-patient count.
Every client instance could launch the same operation.	Workstations and tabs duplicated scans and updates.
No shared lock or completed-run cooldown existed.	Repeated work could consume the pool.

The batching commit was reasonable for its stated objective—preventing Nginx/Kong request-line overflow—but it treated the transport symptom rather than the maintenance-job architecture. The outage arose from the interaction of increased request count, read-path triggering, and missing concurrency control.

7. Corrective Design

Control	Implementation	Effect
Atomic RPC	apply_auto_onp_patient_type(UUID)	One REST call and one database UPDATE replace scan-plus-many-PATCH behavior.
Database lock	pg_try_advisory_xact_lock per branch	Only one branch refresh runs; duplicates return immediately.
Least privilege	SECURITY INVOKER	The function cannot bypass existing caller permissions.
Client single-flight	Per-branch in-flight promise	Concurrent calls in one client share one operation.
Client cooldown	Five minutes per branch	Repeated reads do not repeatedly start maintenance.
Compatibility	Legacy batching only if RPC is absent	Allows safe rolling deployment.
Watchdog	Real query every minute; three strikes	Restarts only PostgREST when PostgreSQL is healthy.

8. Recovery and Safety Actions

- Captured host, container, connection, and authentication evidence before service changes.
- Created a root-only logical PostgreSQL backup and checksum before restart.
- Restarted only PostgREST; PostgreSQL, Auth, and Storage were not restarted.
- Restricted Kong, Portainer, Studio, and database/pooler ports to loopback.
- Validated daily backup integrity and changed future backup permissions to root-only.
- Installed a watchdog that tests a real authenticated data query.

9. Validation Evidence

Validation	Result	Evidence
Focused regression tests	PASS	4 of 4 ONP and migration tests passed.
Full automated suite	PASS	104 test files; 483 tests passed.
TypeScript	PASS	No diagnostics.
Production build	PASS	Vite transformed 3,411 modules successfully.
RPC concurrency	PASS	12 simultaneous calls; all HTTP 200; maximum 3.063 seconds.
Connection recovery	PASS	PostgREST returned to six idle connections.
Error recurrence	PASS	No PGRST002/PGRST003 entries after recovery during acceptance checks.
Live business queries	PASS	locations and app_settings returned HTTP 200.
Watchdog	PASS	Enabled, active, executing each minute, and reporting WATCHDOG_OK.

10. Recurrence Risk Assessment

Risk	Before	After	Rationale
Same ONP request storm	High	Low	One RPC, lock, single-flight, and cooldown remove the amplification path.
PostgREST stale state	High	Low-Medium	The watchdog detects real-query failure and performs a narrow restart.
Other high-volume paths	Medium	Medium	Unrelated queries still require concurrency budgets and load testing.
Infrastructure failure	Medium	Medium	Code cannot eliminate host, DNS, storage, network, or provider outages.

Answer to the recurrence question

The corrected architecture is specifically designed to prevent the same automatic-ONP request amplification from exhausting PostgREST again. Code-level prevention and server-side automatic recovery were both validated in production. This is a strong risk reduction, not an absolute guarantee of zero future downtime.

11. Recommended Management Actions

Priority	Action	Owner / cadence
P1	Add external monitoring for authenticated REST business queries, latency, and consecutive HTTP 5xx failures.	Infrastructure / immediate
P1	Rotate any credentials disclosed during incident handling and maintain an approved access-key inventory.	Security / immediate
P1	Test database restoration from backup, not only backup creation.	Infrastructure / monthly
P2	Add multi-client load tests for startup and maintenance features.	Engineering / before release
P2	Require concurrency and request-budget review for writes triggered from read paths.	Engineering / pull requests
P2	Track pool acquisition timeouts, request latency, and status rates on a dashboard.	Infrastructure / ongoing

12. Lessons Learned

- **Fix the architectural cause, not only the transport symptom.** Splitting URLs prevented proxy rejection but increased load; the correct boundary was one atomic operation.
- **Reads should not silently trigger unbounded maintenance writes.** If unavoidable, use deduplication, throttling, locking, and observability.
- **Container health is not service health.** PostgREST was running while real queries returned HTTP 503.
- **Concurrency testing is essential.** Single-client tests did not expose cross-tab and cross-workstation amplification.
- **Narrow recovery reduces risk.** Restarting only PostgREST preserved healthy database, Auth, and Storage services.

13. Evidence and Change References

Reference	Details
Triggering commit	0a107c6d776a7201ffe7f9791dec550ef9e479e0 - Batch automatic ONP patient updates
Corrective commit	1b7bc4deee0b97c37719e402cf37d62e4cf42757 - Prevent concurrent automatic ONP refresh storms
Production error	PGRST003 pool acquisition timeout followed by PGRST002 schema-cache failure.
Migration	supabase/migrations/20260921000000_atomic_auto_onp_refresh.sql
Client correction	services/api.ts - RPC path, compatibility fallback, single-flight, and cooldown.
Regression tests	services/api.autoOnpBatching.test.ts and atomic_auto_onp_refresh.test.ts
Recovery backup	/root/supabase-backups/postgres-pre-rest-restart-20260921-110432.dump

14. Final Status

Item	Status
Client-facing REST service restored	Complete
Database protected by pre-change backup	Complete
Atomic production migration deployed	Complete
Corrective code pushed to origin/main	Complete
Automatic recovery watchdog active	Complete
Post-remediation concurrency validation	Complete

Report limitation: This document reflects evidence available during the live incident response and validation window. It does not replace a forensic disk image, long-duration load test, disaster-recovery exercise, or independent security audit.